

Java vs. JavaScript: Core Concept Notes

A quick reference guide for variable syntax, naming conventions, and type rules.

1. Variable Naming Rules (Identical for Both)

- **Allowed Characters:** Names can contain letters, digits, underscores (`_`), and dollar signs (`$`).
- **Starting Character:** Names must begin with a letter, `$`, or `_`.
- **Whitespace & Case:** Names cannot contain whitespace and are strictly **case-sensitive** (`myVar` and `myvar` are different).
- **Style Convention:** Standard variables should start with a lowercase letter using camelCase.
- **Reserved Words:** Keywords belonging to the languages (like `int` or `boolean`) **cannot** be used as names.

2. Constants & Final Variables

When creating variables whose value should never change:

- **Java:** Uses the `final` keyword (e.g., `final int BIRTHYEAR = 2000;`).
- **JavaScript:** Uses the `const` keyword (e.g., `const BIRTHYEAR = 2000;`).
- **Convention:** By widespread convention, unchanging constants are written in **UPPER_CASE** in both languages. This is not strictly required by the compilers, but it is standard professional practice to maximize readability.

3. Declaring Variables & Types

This is where the two languages branch out fundamentally:

Feature	Java (Static Typing)	JavaScript (Dynamic Typing)
Declaration Syntax	Must explicitly state the type: <code>int x = 5;</code> <code>String name = "Alex";</code>	Uses generic keywords: <code>let x = 5;</code> <code>let name = "Alex";</code>
Built-in Types	Strict types: <code>int</code> , <code>float</code> , <code>boolean</code> , <code>char</code> , <code>String</code> .	Implicitly infers number, string, boolean, etc.
The Single Quote Rule	Single quotes <code>'a'</code> are strictly for the <code>char</code> type, which evaluates as a numeric decimal (ASCII) value.	There is no <code>char</code> type. Single quotes <code>'a'</code> and double quotes <code>"a"</code> do exactly the same thing: create strings.

4. The Concatenation Domino Effect

The Golden Rule: Both languages read code from left to right. If a string is involved anywhere in a chain of `+` operators, the final outcome of that entire line will always result in a single string.

- **Left-to-Right:** `"Result: " + 5 + 2` becomes `"Result: 5" + 2` → `"Result: 52"`.

- **Parentheses Break the Chain:** If you isolate arithmetic inside (), the computer calculates the math first before meeting the string. "Result: " + (5 + 2) → "Result: " + 7 → "Result: 7".